

Dexios: A V1 Format for Encrypting and Storing Sensitive Data

brxken128

March 10, 2026

Abstract

Dexios is an open-source file encryption tool written in Rust. Its current normal format combines a 512-byte canonical V1 header, authenticated streaming encryption, Argon2id password hashing, keyslot-based master-key wrapping, and Dexios-owned manifest-first archive payloads.

1 Introduction

Dexios is aimed at encrypting files and packed directory archives for storage or transfer. The project separates the user-facing CLI, the workflow layer, and the low-level cryptographic primitives into different crates so that format-critical code remains explicit and auditable.

This reference describes the current public format and workflow surface. The source tree and mdBook safety contract remain the authority for implementation details, but this document is kept aligned to those sources.

2 Modes of Operation

This section describes the current Dexios workflows, including encryption, decryption, packing, unpacking, deletion semantics, and hashing.

2.1 Encryption

New encryption uses the LE31 stream object provided by the `aead` crate. Payloads are processed in 1 MiB blocks.

For new files, Dexios:

- generates a random 32-byte master key,
- generates a 20-byte payload nonce,
- generates salts and 24-byte wrapping nonces for keyslots,
- hashes the user-provided key material with Argon2id,
- wraps the master key inside one or more keyslots,
- serializes the canonical V1 header,
- encrypts the payload in stream mode.

The current CLI writes a 512-byte canonical V1 header. The payload AAD covers the 64-byte immutable static header, including the canonical discriminator, schema fields, payload kind, payload framing profile, KDF parameter profile, fixed slot capacity, and payload nonce. The mutable keyslot table is excluded from payload AAD so key operations can update slots without re-encrypting the payload.

2.1.1 File-size Edge Cases

If the plaintext size is exactly divisible by the 1 MiB block size, Dexios still emits a final authenticated last block. This avoids ambiguity at the end of the stream.

2.1.2 Detached Mode

Dexios can store the header separately from the encrypted payload. In this mode, the detached header file contains the exact serialized 512-byte canonical V1 header, and the ciphertext file starts immediately with encrypted payload bytes.

Detached mode is useful when the user wants to keep the header under separate control, but header restoration is generally unsupported unless the target file already contains a zeroed canonical header region at the beginning.

2.2 Decryption

Decryption reads and deserializes the header first.

For canonical V1 files, Dexios iterates through the available keyslots and attempts to derive a wrapping key from the supplied key material and the keyslot salt. If one of the derived keys successfully decrypts the wrapped master key, that master key is used to decrypt the payload stream.

Legacy inspection and compatibility helpers still exist in the codebase, but they are no longer the normal product surface.

2.3 Packing

Packing is an extension of encryption.

The current implementation accepts one or more input directories, derives unique archive roots for them, builds a Dexios-owned manifest-first archive payload, and encrypts that payload with the normal canonical V1 file workflow. Recursive traversal is the default behavior; the `--recursive` flag is retained only as a compatibility alias.

The manifest-first archive payload starts with a **DXAR** manifest and stores file contents as ordered **DXBF** body frames. ZIP bytes, central-directory metadata, ZIP crate types, compression selectors, and broad metadata knobs are not canonical V1 archive format surface.

Normal pack operation has no full plaintext archive temporary file. Pack still materializes the entry list and streams plaintext file bodies through the archive writer while encryption is running, so plaintext exposure remains ordinary process and filesystem exposure for opened source files.

2.4 Unpacking

Unpacking is an extension of decryption.

The source file is decrypted through the authenticated V1 stream reader. Dexios validates the **DXAR** manifest, normalizes archive paths, rejects traversal attempts, rejects duplicate output paths after normalization, rejects unsafe output symlink escapes, and stages selected file bodies from ordered **DXBF** body frames under the chosen directory.

Normal unpack operation has no full plaintext archive temporary file. Unpack side plaintext exposure is scoped to selected staged file bodies and ordinary filesystem temporary/staged files while the workflow is running. Selected outputs are committed only after final stream authentication.

2.5 Deletion Semantics

Dexios no longer exposes secure-erase behavior as a product feature.

Instead, selected workflows can delete source inputs after successful completion:

- `encrypt --delete-input`
- `decrypt --delete-input`
- `unpack --delete-input`
- `pack --delete-source`

If an operation fails, its source inputs are retained. Delete-after-success is ordinary filesystem cleanup, not secure erase or physical sanitization.

2.6 Hashing

BLAKE3 is the only hashing algorithm exposed by Dexios' checksum mode.

This mode is useful for manual or out-of-band verification of files, but it is not the primary integrity mechanism of the Dexios format. Ciphertext and header integrity are already enforced by authenticated encryption and AAD validation.

3 Key Operations

Dexios can modify canonical V1 keyslots because payload AAD excludes the mutable keyslot table. Slot updates remain protected by slot-scoped AAD for each wrapped master key.

3.1 Key Changing

Changing a key decrypts the wrapped master key with the old credential, derives a new wrapping key from the replacement credential, and rewrites the proven physical keyslot with a new salt and nonce.

3.2 Key Addition

Adding a key decrypts the wrapped master key and writes a new wrapped copy of that same master key into the first empty physical keyslot.

The current V1 format allows up to 4 physical keyslots.

3.3 Key Deletion

Key deletion identifies the keyslot that can be opened with the supplied old credential and removes that physical keyslot from the header.

All current V1 key-manipulation operations act on the proven physical keyslot instead of compacting or reordering the keyslot table.

4 Cryptography

4.1 Password Hashing

Dexios uses memory-hard password hashing to derive 32-byte wrapping keys from user-supplied key material.

Current canonical V1 writes use a 16-byte salt and Argon2id (RFC 9106) with the frozen parameter profile defined in `dexios-core/src/kdf.rs`: `m_cost 262144` KiB (256 MiB), `t_cost 4` passes, `p_cost 4` lanes, 32-byte output, and Argon2 version `0x13`.

4.1.1 Hashing Algorithms

The normal write policy is Argon2id only. New canonical V1 files do not expose alternate KDF selection or user-configurable KDF parameters.

The historical Argon2id tag `[0xDF, 0x02]` may be recognized as unsupported metadata for explicit diagnostics, but it is not used for current derivation and is not a normal write policy.

4.1.2 Performance

The KDF parameters are intentionally expensive compared to a general-purpose password hash, but still chosen to remain practical on normal user hardware. A single Argon2id derive costs roughly 0.5 seconds on reference hardware; the four lanes are computed sequentially in pure Rust, but the digest is spec-correct. In most real workflows, storage I/O dominates total runtime once the key has been derived.

4.2 Encryption Algorithms

The current CLI supports one suite for new output:

- XChaCha20-Poly1305
- LE31 stream mode

4.3 Threat Model

Dexios is designed to protect file contents and format integrity at rest and in transit. It does not attempt to defend against a fully compromised host that can observe the running process directly.

4.4 Key Inputs

Dexios currently accepts key material from:

- an interactively entered passphrase,
- a keyfile,
- a generated passphrase,
- or the `DEXIOS_KEY` environment variable.

The current CLI resolves these sources in this order:

1. explicit keyfile,
2. explicit generated passphrase request,
3. environment variable,
4. interactive prompt.

4.4.1 Passphrase Autogeneration

The current implementation generates `n` random words from a bundled wordlist and joins them with `-`.

The default is 7 words:

word-word-word-word-word-word-word

4.5 Cryptographic Hygiene

Dexios uses the `zeroize` crate and a `Protected` wrapper to reduce accidental exposure of secrets in memory.

4.5.1 Protected Wrapper

The `Protected` wrapper provides explicit access through `.expose()`, redacts debug output, forbids implicit copying, and zeroizes the wrapped value on drop.

4.6 RNGs

Random values are generated through the current `rand` API, which draws from operating system entropy sources.

5 Headers

The current V1 format uses a 512-byte canonical V1 header.

5.1 Header Structure

The V1 layout is:

```
{StaticHeader | KeyslotArea}
```

The static header is the 64-byte immutable static header:

- 4-byte magic: `DXIO`
- 2-byte product version field: `0x0001`
- 4-byte canonical discriminator: `CV1\0`
- 1-byte schema profile
- 1-byte payload kind
- 1-byte payload framing profile
- 1-byte KDF parameter profile
- 1-byte fixed keyslot capacity
- 1 reserved byte
- 20-byte payload nonce
- zeroed reserved bytes through the end of the static header

Together, the canonical prefix is `DXIO 00 01 CV1\0`.

5.2 Keyslots

The V1 keyslot area contains space for 4 fixed physical slots. Each slot is a 112-byte physical keyslot. Empty slots serialize as all-zero records. Active slots stay in their physical position; key operations do not compact or reorder slots.

Each active keyslot contains:

```
{State | PhysicalSlotIndex | KdfProfile | KdfParamProfile | Salt | Nonce |  
WrappedMasterKey | Padding}
```

The wrapped master key is 48 bytes long.

The normal KDF identifier is `[0x01, 0x01]` for Argon2id. The historical Argon2id tag `[0xDF, 0x02]` is unsupported metadata for diagnostics, not a current derivation path.

5.3 AAD

For V1, payload AAD covers the 64-byte immutable static header. This protects the magic, version, canonical discriminator, schema/profile bytes, payload nonce, and fixed slot-capacity metadata while still allowing keyslot updates without re-encrypting the payload.

Wrapped master keys use slot-scoped AAD that binds the immutable static header, physical slot index, KDF profile id, KDF parameter profile id, salt, and keyslot nonce.

5.4 Header Operations

Dexios supports:

- stripping embedded headers by zeroing the serialized canonical header region,
- dumping valid embedded canonical headers,
- restoring headers only when the target starts with enough zero bytes for the canonical header.

Files created in detached-header mode generally do not reserve enough zero space for restoration.

6 Conclusion

Dexios' current design favors a compact canonical V1 byte contract, explicit keyslot handling, modern authenticated encryption, and a relatively small implementation surface. The project is best understood as a practical at-rest file-encryption format rather than a general-purpose secure messaging or secret-management system.

References

1. <https://csrc.nist.gov/projects/block-cipher-techniques/bcm>
2. <https://research.nccgroup.com/2020/02/26/public-report-rustcrypto-aes-gcm-and-chacha20-poly1305-implementation-review/>
3. <https://www.eff.org/document/passphrase-wordlists>
4. <https://eprint.iacr.org/2016/027.pdf>